

[Algorithm]

기수 정렬 알고리즘

인하대학교 소프트웨어융합공학과

김종훈 교수



인하대학교
INHA UNIVERSITY

기수 정렬

□ 기수 정렬 (Radix Sort)

- 비교 정렬이 아니고, 숫자를 부분적으로 비교하는 정렬 방법이다.
- 기 (radix)는 특정 진수를 나타내는 숫자들이다.
 - 예를 들어, 10진수의 기는 0, 1, 2, ..., 9이고, 2진수의 기는 0, 1이다.
- 기수 정렬은 제한적인 범위 내에 있는 숫자에 대해서 각 자릿수 별로 정렬하는 알고리즘이다.
- 기수 정렬의 가장 큰 장점은 어느 비교 정렬 알고리즘보다 빠르다.

기수 정렬 아이디어

입력	1의 자리	10의 자리	100의 자리
089	07 0	9 1 0	0 35
070	91 0	1 3 1	0 70
035	13 1	0 3 5	0 89
131	03 5	0 7 0	1 31
910	08 9	0 8 9	9 10

기수 정렬 아이디어

- 예제와 같이 5개의 3자리 십진수가 입력으로 주어지면
 - 가장 먼저 각 숫자의 1의 자리만 비교하여 작은 수부터 큰 수를 정렬한다.
 - 그 다음엔 10의 자리만을 각각 비교하여 정렬한다. 이때 반드시 지켜야 할 순서가 있다.
 - 예제에서 10의 자리가 3인 131과 035가 있는데, 10의 자리에 대해 정렬될 때 131이 반드시 035 위에 위치하여야 한다.
 - 10의 자리가 같은데 왜 035가 131 위에 위치하면 안 되는 것일까?
 - 그 답은 1의 자리에 대해 정렬해 놓은 것이 아무 소용이 없게 되기 때문이다.

기수 정렬

□ 정렬 알고리즘의 안정성(Stability)

- 입력에 중복된 숫자가 있을 때, 정렬된 후에도 중복된 숫자의 순서가 입력에서의 순서와 동일하면 정렬 알고리즘이 **안정성 (stability)**을 가진다고 한다.

기수 정렬

□ 아래의 예에서 2개의 10이 입력에 있을 때

- **안정한 정렬 (stable sort)** 알고리즘은 중복된 숫자에 대해 입력에서 앞서 있던 숫자가 정렬 후에도 앞서 있고,
- 불안정한 정렬 알고리즘은 정렬 후에도 그 순서가 반드시 지켜지지 않는다.

정렬 전	90 A	10 B	35 C	13 D	10 E	35 F	31 G	08H
안정한 정렬	08H	10 B	10 E	13 D	31 G	35 C	35 F	90 A
불안정한 정렬	08H	10 E	10 B	13 D	31 G	35 F	35 C	90 A

RadixSort 알고리즘

입력: n 개의 r 진수의 k 자리 숫자

출력: 정렬된 숫자

1. for $i = 1$ to k
2. 각 숫자의 i 자리 숫자에 대해 안정적인 정렬을 수행한다
3. return 배열 A

RadixSort 알고리즘

□ Line 1

- for-루프에서는 1의 자리~ k자리까지 차례로 안정한 정렬을 반복한다.

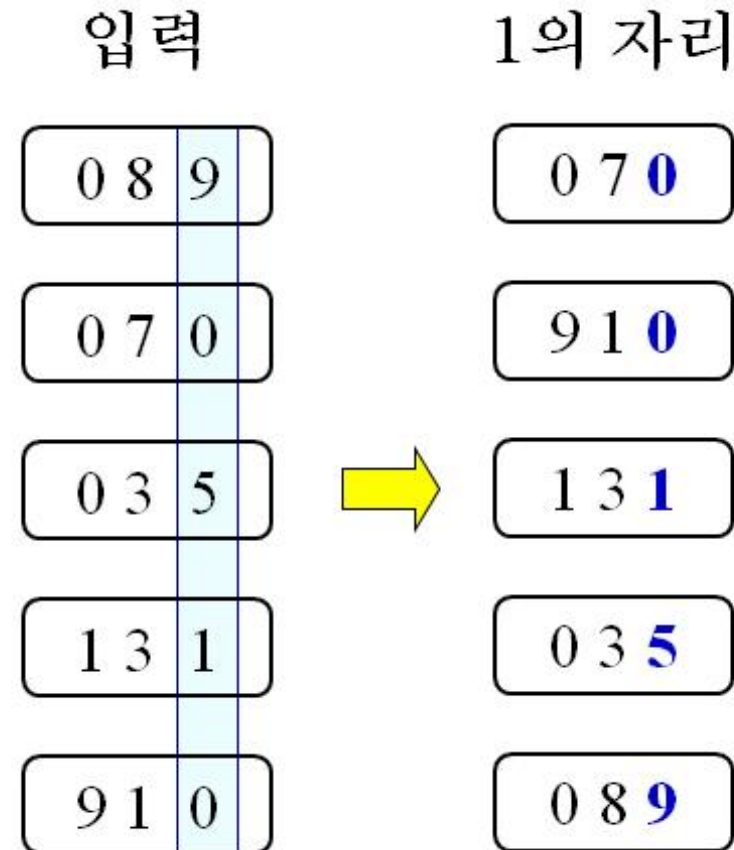
□ Line 2

- 각 숫자의 i자리 수만에 대해 다음과 같이 정렬한다. 입력 숫자가 r진수라면, i자리가
 - '0'인 수의 개수
 - '1'인 수의 개수
 - ...
 - '(r-1)'인 수의 개수를 각각 계산하여
- i자리가 '0'인 숫자로부터 '(r-1)'인 숫자까지 차례로 안정성에 기반을 두어 정렬한다.

RadixSort 알고리즘

□ 예제에서 $i=1$ 일 때

- 입력의 각 숫자의 1의 자리 수만을 보면, 9, 0, 5, 1, 0이므로, 1의 자리가 '0'인 숫자가 2개, '1'인 숫자가 1개, '5'인 숫자가 1개, '9'인 숫자가 1개이다.
- 따라서 1의 자리가 '0'인 숫자 070과 910, '1'인 숫자 131, '5'인 숫자 035, 마지막으로 '9'인 숫자 089 순으로 입력의 숫자들이 정렬된다.



구현코드

```
def lsd_sort(a):
    WIDTH = 3 # 스트링의 문자 수
    N = len(a) # 입력 크기
    R = 128 # radix 크기
    temp = [None] * N
    for d in reversed(range(WIDTH)): # d번째 문자 처리, d = 2, 1, 0
        count = [0] * (R+1)
        for i in range(N): # 빈도수 계산
            count[ord(a[i][d])+1] += 1 # ord() 인자의 unicode 값 리턴
        for j in range(1, R): # 빈도수 누적 계산
            count[j] += count[j-1]
        for i in range(N): # data 복사
            p = ord(a[i][d])
            temp[count[p]] = a[i]
            count[p] += 1
        for i in range(N): # t를 a로 복사
            a[i] = temp[i]
        print("%d번째 문자: %s" % d, end="")
        for x in a: print(x, ' ', end="")
        print()
    a = ['ICN', 'SFO', 'LAX', 'FRA', 'SIN', 'ROM', 'HKG', 'TLV',
          'SYD', 'MEX', 'LHR', 'NRT', 'JFK', 'PEK', 'BER', 'MOW']
    print("정렬 전: %s" % a)
    for x in a: print(x, ' ', end="")
    print()
    lsd_sort(a)
```

시간복잡도

□ 먼저 for-루프가 d 번 반복된다.

- 단, d 는 입력 숫자의 최대 자릿수이다.
- 한번 루프가 수행될 때 n 개의 숫자의 i 자리 수를 읽으며, r 개로 분류하여 개수를 세고, 그 결과에 따라 숫자가 이동하므로 $O(n+r)$ 시간이 걸린다.

□ 따라서 시간복잡도는 $O(d(n+r))$ 이다.

- 여기서 d 나 r 이 입력 크기인 n 보다 크지 않으면, 시간복잡도는 $O(n)$ 이 된다.

기수 정렬

□ LSD 기수 정렬

- RadixSort는 1의 자리부터 k자리로 진행하는 경우, Least Significant Digit (LSD) 기수 정렬 또는 RL (Right-to-Left) 기수 정렬이라고 부른다.

□ MSD 기수 정렬

- k자리부터 1의 자리로 진행하는 방식은 Most Significant Digit (MSD) 기수 정렬 또는 LR (Left-to-right) 기수 정렬이라고 부른다.

응용

- 기수 정렬은 계좌 번호, 날짜, 주민등록번호 등으로 대용량의 상용 데이터베이스 정렬, 랜덤 128 비트 숫자로 된 초대형 파일 (예, 인터넷 주소)의 정렬, 지역 번호를 기반한 대용량의 전화 번호 정렬에 매우 적절하다.
- 또한 다수의 프로세서들이 사용되는 병렬 (Parallel) 환경에서의 정렬 알고리즘에 기본 아이디어로 사용되기도 한다.